SORTING ALGORITHMS

UNIT-3: DATA STRUCTURES (ETCCDS202)

Submitted by:

Aryan Solanki
Roll No: 2501730347
School of Engineering & Technology
B.Tech (AI/ML)
Section-A

Experimental Results - Execution Times (ms)

All experiments were run using Python time.perf_counter() for high-resolution timing. Each algorithm sorted a copy of the dataset to ensure fair comparison. Random seed = 42 was used for reproducibility across all random datasets.

Size	Input Type	Insertion Sort (ms)	Merge Sort (ms)	Quick Sort (ms)
1,000	Random	48.8569	3.0813	2.1237
1,000	Sorted	0.2583	2.5188	87.0281
1,000	Reverse	78.4160	2.7936	66.5272
5,000	Random	660.4913	10.1797	5.7491
5,000	Sorted	0.9870	19.0161	2123.1197
5,000	Reverse	1097.6318	6.7822	939.6057
10,000	Random	3678.5264	47.8381	26.4615
10,000	Sorted	2.3314	34.3939	7735.6739
10,000	Reverse	6926.8389	22.3581	5750.7577

Comparative Analysis by Input Type

Random Input

On random data, Quick Sort is fastest across all sizes — 2.12 ms at n=1,000 and 26.46 ms at n=10,000 — owing to good pivot placement on average and superior cache locality from in-place swaps. Merge Sort is close behind. Insertion Sort is the clear loser: 48.86 ms at n=1,000 escalating to 3,678 ms at n=10,000, a 75x increase for a 10x input size — exactly the O(n^2) signature.

Already-Sorted Input

Insertion Sort delivers its best-case O(n) performance here: 0.26 ms at n=1,000 and only 2.33 ms at n=10,000. No element ever needs to shift, so the inner while loop executes zero times for every element. Merge Sort still runs at O(n log n) regardless of input order (34.39 ms at n=10,000). Quick Sort, however, hits its worst case: 87.03 ms at n=1,000 and a catastrophic 7,735 ms at n=10,000. With last-element pivot on sorted input, every partition produces one empty subarray and one subarray of size n-1, giving O(n^2) behaviour.

Reverse-Sorted Input

Reverse-sorted input is the worst case for Insertion Sort — every new element must shift past all previously inserted elements: 78.42 ms at n=1,000 reaching 6,926 ms at n=10,000, almost double the random-input time. Merge Sort stays stable at 22.36 ms at n=10,000 — it does not care about input order. Quick Sort also degrades significantly (5,750 ms at n=10,000) because the last element is always the minimum on reverse-sorted data, again producing maximally unbalanced partitions.

O(n^2) vs O(n log n) - Theory vs Experiment

Increasing input size from n=1,000 to n=10,000 is a 10x increase. Theory predicts:

- O(n^2) algorithm: should run ~100x longer.

- O(n log n) algorithm: should run ~11x longer (log10000/log1000 = 1.33, so 10 x 1.33 = 13.3x approx.).

- O(n) algorithm: should run ~10x longer.

Size	Input Type	1,000 ms	10,000 ms	Actual Ratio	Theory
Insertion Sort	Random	48.86	3678.53	~75x	O(n^2) ~100x
Insertion Sort	Sorted	0.26	2.33	~9x	O(n) ~10x
Insertion Sort	Reverse	78.42	6926.84	~88x	O(n^2) ~100x
Merge Sort	Random	3.08	47.84	~15.5x	O(n log n) ~13x
Merge Sort	Sorted	2.52	34.39	~13.6x	O(n log n) ~13x
Quick Sort	Random	2.12	26.46	~12.5x	O(n log n) ~13x
Quick Sort	Sorted	87.03	7735.67	~89x	O(n^2) ~100x worst

The experimental ratios closely match theoretical predictions. Minor deviations from exact theoretical values are expected due to CPU cache effects, Python interpreter overhead, memory allocation time, and constant factors hidden inside Big-O notation. The key patterns are preserved: O(n^2) algorithms scale drastically worse than O(n log n) as input size grows.

Quick Sort - Worst Case Deep Dive

Quick Sort with Lomuto partition (last element as pivot) has a well-known vulnerability on sorted and reverse-sorted inputs. The observed times make this concrete:

Input Type	N = 1,000	N = 5,000	N = 10,000	Behaviour
Random	2.12 ms	5.75 ms	26.46 ms	O(n log n) avg
Sorted	87.03 ms	2123.12 ms	7735.67 ms	O(n^2) worst
Reverse	66.53 ms	939.61 ms	5750.76 ms	O(n^2) worst

On sorted input at n=10,000, Quick Sort takes 7,735 ms vs Merge Sort's 34 ms - that is 227x slower. The root cause: with last-element pivot on sorted data, every single partition produces one subarray of size 0 and one of size n-1. This forces n levels of recursion instead of log n, giving O(n^2) total comparisons. The recursion depth also hits the Python default limit, which is why sys.setrecursionlimit(50000) is required. Fix strategies: (1) Randomized pivot - pick a random element as pivot, statistically eliminates worst case. (2) Median-of-three - use median of first, middle, last elements as pivot. (3) Introsort - Python's own sort switches from Quick Sort to Heap Sort when recursion depth exceeds 2*log(n), guaranteeing O(n log n) worst case.

Algorithm Properties - Stability and Memory

Quick Sort with Lomuto partition (last element as pivot) has a well-known vulnerability on sorted and reverse-sorted inputs. The observed times make this concrete:

Algorithm	Stable	In-Place	Extra Space	Best	Average	Worst
Insertion Sort	Yes	Yes	O(1)	O(n)	O(n^2)	O(n^2)
Merge Sort	Yes	No	O(n)	O(n log n)	O(n log n)	O(n log n)
Quick Sort	Yes	Yes	O(log n)	O(n log n)	O(n log n)	O(n^2)

Stable means equal elements preserve their original relative order after sorting. Insertion Sort and Merge Sort are stable because equal elements are never unnecessarily swapped. Quick Sort is unstable because the partition step can swap equal elements across the pivot boundary.

In-place means only O(1) or O(log n) extra memory is used beyond the input. Merge Sort is out-of-place because the merge step creates new left and right sublists at each level of recursion, totalling O(n) extra space. This is a real cost in memory-constrained environments.